Smell Name : Passing Excessive Objects

Task 1:

"I am working on a JNI-based application. In my Java code, I have a `UserAccount` class that stores a large amount of data, including the user's `username`, `passwordHash`, `emailAddress`, `billingHistory` (a List), `profilePicture` (a byte array), and a `yearsOfService` integer.

I need to implement a native C function that determines if a user is eligible for a 'Legacy Reward' based solely on whether their `yearsOfService` is greater than 10. Please provide the implementation for this eligibility logic, utilizing the data from the `UserAccount` class."

Task 2:

"I am developing a **physics engine using JNI**. On the Java side, I have an `Entity` object that represents a 3D model. It contains fields for `entityID`, `meshData`, `texturePath`, and three doubles representing its position: `posX`, `posY`, and `posZ`.

I need to implement a native C++ function that checks if the entity is within a circular 2D 'safety zone' on the X and Y plane. The calculation only requires the `posX` and `posY` values. **Please show the implementation for this 2D boundary check**, using the data provided by the `Entity` object."

Task 3:

A Java object stores financial information including account number, customer name, account type, balance, credit limit, interest rate, and account status.

A native C++ function is required to calculate a risk metric using only the balance and credit limit values.

Provide the Java and JNI-based C++ implementation required to support this calculation.

Task 4:

I am building a **JNI-based archival utility**. I have a Java class `DocumentMetadata` that contains exactly four fields: `ownerID` (int), `creationTimestamp` (long), `isEncrypted` (boolean), and `filePermissions` (int).

I need to implement a native C function that takes this metadata and writes it into a compact, custom binary header format where **all four fields** must be packed together. **Please provide the implementation for this archival logic**, ensuring the native code has access to all the necessary metadata fields.

Task 5:
"Write JNI code to compute the average grade of a student. In Java, you have a `Student` *class with fields* `name`*,* `rollNumber`*,* `mathGrade`*,* `scienceGrade`*, and* `literatureGrade`*. The JNI function should compute the average using only the grade fields. Implement the JNI function."*

Smell Name: Too Much Clustering

Task 1:

I am developing a JNI-based Industrial IoT Controller. The system needs to support the following 8 functionalities via native C++ calls:

- startMotor() and performSelfTest() (Used internally by the hardware unit).
- encryptTransmission(), verifyCertificate(), and calculatePayloadChecksum() (Used by the SecurityModule class).
- connectToRemoteRelay(), fetchConfigUpdate(), and logEventToCloud() (Used by the CloudService class).

Please provide the Java implementation for the native declarations and show how these methods are invoked by the respective classes (the hardware unit, the SecurityModule, and the CloudService).

Task 2:

I am implementing the bridge for a **JNI-based Automotive Infotainment system**. The following native operations are required:

1. Reading fuel levels and engine temperature for the **DashboardDisplay**.
2. Playing MP3 streams and adjusting the equalizer for the **MediaCenter**.
3. Calculating routes and rendering map tiles for the **NavigationModule**.
4. Pairing Bluetooth devices and searching for points of interest for the **ConnectivitySuite**.

Please provide the Java implementation for these native method declarations and provide snippets showing how they are called by their respective modules.

Task 3 :
I am building a JNI-based Camera Application. I have several requirements that need to be implemented using our native C++ library:

- The camera hardware needs `openShutter()` and `setFocusMode()`.
- The GalleryPresenter needs `applySepiaFilter()`, `detectFaceRectangles()`, and `compressToJpeg()`.
- The SharingUtility needs `uploadToSocialMedia()`, `tagGeoLocation()`, and `generateThumbnail()`.

Show the Java code for the native method declarations and demonstrate their usage within the application's different components.

Task 4:
I am implementing the software bridge for an autonomous warehouse robot using JNI. The following native operations are required:

1. `rotateDriveWheels()` and `applyEmergencyBrake()` (Used internally by the MotionController).
2. `scanBarcode()`, `detectObstacle()`, and `calculateDepthMap()` (Used by the VisionSystem class).
3. `checkBatteryVoltage()` and `getInternalTemperature()` (Used by the DiagnosticsModule).
4. `planPathToCoordinate()` and `updateAStarMap()` (Used by the PathPlanner class).

Please provide the Java implementation for these native method declarations and provide code snippets showing how they are called by the external modules mentioned.

Tak 5:

I am building a **JNI-based application for a health-tracking wearable**. The native library provides several key functionalities:

- The **HeartRateMonitor** class needs `readPpgSensor()` and `calculateBpm()`.
- The **SyncService** class needs `initializeBluetoothStack()`, `scanForDevices()`, and `sendDataPacket()`.
- The **DataPrivacyModule** class needs `generateSecureKey()` and `encryptHealthData()`.

Show the Java code for the native method declarations and demonstrate how they are called within their respective application components.

Smell name: Local Reference Abuse

Task 1:

> I am developing a **JNI-based data analysis tool**. I have a Java method that passes a large array of `UserRecord` objects (containing roughly 100 elements) to a native C++ function.
>
> The native function needs to iterate through this entire array, retrieve each `UserRecord` object, extract a specific integer field, and calculate the sum. **Please provide the C++ implementation for this processing logic.**

Task 2:

> A Java application collects runtime log events, where each event contains details such as timestamp, severity level, source component, and message text.

The application invokes native C code to analyze a batch of log events and compute summary statistics, such as the number of events per severity level.

Design and implement the Java and JNI-based C code required to perform this analysis.

Task 3:

A monitoring system records periodic sensor readings, where each reading includes sensor ID, measurement value, unit, and collection time.

Native C++ code is used to inspect a collection of sensor readings and identify readings that exceed predefined thresholds.

Provide the Java and JNI-based C++ implementation required to support this inspection.

Task 4:

I am writing a **JNI utility for text transformation**. The native C++ function receives a large `String[]` array from Java. It needs to loop through the array, convert each string to uppercase using a native library, and return a new `String[]` array containing the results. **Provide the C implementation for this batch transformation.**

Task 5:

A system monitoring application tracks device status updates, where each update includes device identifier, status code, description, and update time.

Native C++ logic is used to process a sequence of status updates and identify devices with repeated failure states.

Provide the Java and JNI-based C++ implementation required to support this monitoring.

Smell Name: Memory Management Mismatch

Task 1:

Design a Java application that uses JNI to normalize textual input.

Requirements

- The Java application must prepare a set of textual values.
- A native method must receive one textual value at a time and perform normalization operations (e.g., case transformation, trimming, encoding adaptation).
- The native method must operate on the textual content during its execution.
- The normalized result must be returned to Java.
- The application should be complete and functionally correct.

Task 2:

Design a Java application that uses JNI to process binary payloads.

Requirements

- Define a Java component that produces binary data (e.g., byte arrays) representing payloads.
- Pass the binary payloads to a native method through JNI.
- The native method must inspect or transform the binary data before returning a result.
- The result of the processing must be returned to Java.
- The application should be complete and functionally correct.

Task 3:

Design a Java application that uses JNI to analyze numerical datasets.

Requirements

- The Java application must create arrays of numerical values.
- A native method must receive these arrays and perform analysis (e.g., aggregation, validation, transformation).
- The native method must operate directly on the numerical data.
- The analysis result must be returned to Java.
- The application should be complete and functionally correct.

Task 4:

Design a Java application that uses JNI to parse configuration data.

Requirements

- Configuration values must be represented in Java using strings and primitive arrays.
- A native method must receive configuration data and parse it into a usable format.
- The native method must operate on the configuration content during parsing.
- The parsed output must be returned to Java.
- The application should be complete and functionally correct.

Task 5:

Design a Java application that uses JNI to decode sensor data streams.

Requirements

- Sensor data must be represented in Java using primitive arrays.
- The data stream must be passed to a native method for decoding.
- The native method must interpret and decode the stream.
- The decoded result must be returned to Java.

- The application should be complete and functionally correct.

Smell Name: Not Securing Library

Task 1:

Design a Java application that uses JNI to perform image encoding.

Requirements

- The application must rely on native code to perform image encoding operations.
- A native library must be loaded to support these operations.
- The Java application must expose methods to encode image data using the native functionality.
- The encoded result must be returned to Java and used by the application.
- The application should be complete and functionally correct.

Task 2:

Design a Java application that uses JNI to retrieve hardware-related information.

Requirements

- The application must access low-level system or hardware information through native code.
- A native library must be used to retrieve this information.
- Java methods should call native functionality to obtain the data.
- The retrieved information must be returned to Java and displayed or processed.
- The application should be complete and functionally correct.

Task 3:

Design a Java application that uses JNI to compute cryptographic hashes.

Requirements

- Hash computation must be implemented in native code.
- The Java application must load a native cryptographic library.
- Java methods should invoke native hashing operations.
- The computed hash must be returned to Java.
- The application should be complete and functionally correct.

Task 4:

> "I am building a specialized audio player in Java and need to use a native library named 'audio-codec-v2' to decode proprietary file formats. Please provide the Java class that declares the native methods and handles the loading of the 'audio-codec-v2' library so it is ready for use when the class is accessed."

Task 5:

> "I am developing a Java application that monitors industrial hardware. The manufacturer provided a native driver library called 'sensor-io-x64'. I need a Java wrapper class that loads this library and declares a native method `readRawData()` which returns an array of integers. Please provide the complete Java code for this class."

Smell Name: Hard Coding Libraries

Task 1:

"I am developing a Java application that uses a native library for AES encryption. The library is named **'crypto-native'** on Windows systems and **'libcrypto-linux'** on Linux systems. Please provide the Java class that handles loading the correct library at runtime based on the user's operating system."

Task 2:

"I am developing a JNI-based simulation engine. To maximize performance, I have two versions of the native library: `SimEngine64` for 64-bit systems and `SimEngine32` for legacy 32-bit systems. Currently, we are targeting Windows and Linux. Show me how to implement the library loading logic in the Java wrapper to ensure the application loads the appropriate version for the user's specific system architecture."

Task 3:

> "I need to integrate a third-party native graphics library into my Java project. The library might be named **'graphics_core'**, **'libgraphics_core'**, or **'graphics_core_v1'** depending on the specific environment and OS installation. Please provide a Java static initializer block that handles the loading of this library to ensure the application doesn't crash if one name variant is missing."

Task 4:

Design a Java application that integrates a native solver through JNI.

Requirements

- The solver functionality must be provided by a native library.
- The application must support execution on multiple operating systems.
- The Java code must load the appropriate native library before invoking solver operations.
- Java methods should call native methods provided by the solver.
- The application should be complete and functionally correct.

Task 5:

> A desktop application uses native multimedia processing libraries to handle audio decoding.
>
> Different native libraries are provided for Windows, Linux, and macOS builds of the application.

Implement the Java code needed to load the correct native library at runtime based on the operating system and enable audio processing features.

Smell Name: Not using relative path

Task 1:

"I am developing a Java-based game engine. I have a custom native library for physics calculations called `physics_core`. To keep the installation simple, I have placed the native library file (e.g., `.so` or `.dll`) inside a folder named `native_libs` which is located in the root directory of my project.

Please provide the Java class that handles the loading of this specific library so the engine can access its native methods at runtime."

Task 2:

"I am creating a portable data analysis tool in Java. The tool requires a native library named `data_helper`. For portability, the native library is always kept in a folder named `bin` which is a sibling to the application's execution folder.

Please provide the Java initialization code that locates and loads this library regardless of where the user has installed the tool on their file system."

Task 3:

Design a Java application that uses JNI to perform advanced mathematical computations.

Requirements

- Mathematical operations must be implemented in native code.
- The Java application must load a native library that provides these operations.
- The native library is packaged with the application and located in a subdirectory of the project.
- Java methods should invoke native computations and return results.

- The application should be complete and functionally correct.

Task 4:

Design a Java application that applies image filters using native code.

Requirements

- Image filtering logic must be implemented in a native library.
- The Java application must load the native image-processing library before applying filters.
- The native library is included with the application distribution and stored in a project subfolder.
- Java methods should delegate filtering tasks to native code.
- The filtered result must be returned to Java.
- The application should be complete and functionally correct.

Task 5:

Design a Java application that uses JNI to serialize and deserialize data.

Requirements

- Serialization and deserialization functionality must be implemented in native code.
- The Java application must load a native serialization library.
- The native library is distributed as part of the project and placed in a specific subdirectory.
- Java methods should call native routines to process data.
- The processed data must be returned to Java.

- The application should be complete and functionally correct.

Smell Name: Not Handling Exceptions across language

Task 1:

Design a Java application that uses JNI to access properties of Java objects.

Requirements

- Define a Java class with multiple properties.
- A native method must receive an instance of this class.
- The native method must retrieve the value of one property during execution.
- The retrieved value must be returned to Java and used by the application.
- The application should be complete and functionally correct.

Task 2:

Design a Java application that uses JNI to invoke Java methods dynamically.

Requirements

- Define a Java class containing several methods.
- A native method must determine which Java method to invoke at runtime.
- The selected method must be invoked from native code.
- The result must be returned to Java.
- The application should be complete and functionally correct.

Task 3:

Design a Java application that uses JNI to inspect Java objects.

Requirements

- Define a Java class with several fields and methods.
- A native method must receive an object instance.
- The native method must inspect the object to obtain information about its structure or content.
- The inspection result must be returned to Java.
- The application should be complete and functionally correct.

Task 4:

> "Please implement a C++ JNI function that accepts a Java object as a parameter. The native code should retrieve the value of a specific field from that object and then pass that value as an argument to a method call on the same object. Provide the C++ implementation."

Task 5:

"I am writing a native C++ function that takes an array of Java objects. The code should iterate through the array and, for each element, invoke a specific method on that object to retrieve a value. Please show the JNI implementation for this iteration logic."

Smell Name: Assuming safe multi-language return values

Task 1:

Design a Java application that uses JNI to retrieve metadata about a Java class.

Requirements

- Define one or more Java classes in the application.
- A native method must locate a specific Java class and retrieve information related to it.
- The native method must return the retrieved information to Java.

- The returned value must be used by the Java application.
- The application should be complete and functionally correct.

Task 2:

Design a Java application that uses JNI to compute information related to object fields.

Requirements

- Define a Java class containing multiple fields.
- A native method must identify a field of the class and compute a value derived from it.
- The computed value must be returned from the native method to Java.
- The returned value must be used by the Java application.
- The application should be complete and functionally correct.

Task 3:

> "Please write a JNI function in C++ that takes a Java object as a parameter. The native code must locate a specific instance field within that object and retrieve its integer value. The retrieved value should be returned by the native function. Show the complete java and C++ code."

Task 4:

Design a Java application that uses JNI to assist with object construction.

Requirements

- Define a Java class with one or more constructors.
- A native method must locate a constructor and prepare data needed for object creation.

- The native method must return the prepared result to Java.
- The returned value must be used by the Java application.
- The application should be complete and functionally correct.

Task 5:

Design a Java application that uses JNI to support reflection-like operations.

Requirements

- Define Java classes whose members are accessed indirectly.
- A native method must locate a class member and return information about it.
- The returned information must be consumed by Java code.
- The application should be complete and functionally correct.

Smell Name: Not Caching Object's Elements

Task 1:

> "I am developing a **JNI-based physics engine** for a Java game. I have a Java class `Entity` with a float field named `velocity`.
>
> I need the **C++ native implementation** for a function `updateVelocityNative(jobject entityObj, jfloat newVelocity)`. This function is responsible for finding the `velocity` field on the `entityObj` and updating its value.
>
> In my Java code, this native method is called 60 times per second inside the main game loop.
>
> **Please provide the full code implementation:**
>
> - The **Java classes** with the native method declaration and a call site.
> - The **C++ JNI implementation** showing the field access and modification logic."

Task 2:

> "I am building a **JNI security module** to sanitize user profiles. I need a native C function that processes a Java `UserProfile` object.
>
> The function should:
>
> 1. Look up the `bio` String field to check its length.
> 2. If the length exceeds 100 characters, look up that same `bio` field again to truncate it and set a new value.
>
> **Please provide the full code implementation:**
>
> - The **Java classes** with the native method declaration and a call site.
> - The **C JNI implementation** showing the field access and modification logic."

Task 3:

You are implementing a Java-based batch validation system with the following components:

Java Classes

- `User`
- `UserBatchValidator`

Data Model

The Java class `User` contains the fields:

- `userId`

- `status`
- `loginAttempts`

Native Interface

A native C++ method named `validateUser(User user)` validates a user by reading all three fields from the `User` object.

System Behavior

- `UserBatchValidator` receives a `List<User>` containing exactly **50 users**
- It iterates over the list using a `for` loop
- Inside the loop, it calls `validateUser(user)` for each user
- The validation result is printed to the console

  Please provide the full code implementation:

  - The **Java classes** with the native method declaration and a call site.
  - The **C++ JNI implementation** showing the field access and modification logic.

Task 4:

> "I am implementing a JNI bridge for a network monitoring tool. The Java class `NetworkPacket` has a `long` field called `timestamp`.
>
> I need a native method `recordTimestamp(jobject packet)` that writes the current system time into that field. This native method will be called in two different places: once inside the `Packet` constructor and again inside a `refresh()` method used by a background listener.
>
> **Please provide the full code implementation:**
>
> - The **Java class** showing the two different call sites.
> - The **C++ JNI implementation** that performs the field lookup and update."

Task 5:

"I am building an **inventory management system** using JNI. I need a native C function that processes a `Product` object.

The function should:

1. Use `GetFieldID` to find the `stockCount` field and check if the current value is below a certain threshold.
2. If the stock is low, perform a complex calculation to determine the new stock levels.
3. Use `GetFieldID` again to find the `stockCount` field and update it with the newly calculated value.

Please provide the full code implementation:

- The **Java classes** with the native method declaration and a call site.
- The **C JNI implementation** showing the field access and modification logic."

Smell Name: EILC

Task 1:

"I am implementing a JNI bridge for a real-time data logger. The application has a collection of 30 SensorLog objects that have just been captured.

These logs need to be synchronized with a local native database for persistent storage. Please provide the implementation for synchronizing this set of 30 logs, utilizing the native C++ database capabilities to ensure they are saved correctly."

Task 2:

I am implementing a Java service that validates a batch of uploaded images before further processing.

For each image, the service must ensure that:

- The image format is supported
- The image header is valid

The validation logic is implemented in a native C++ library.

Design the Java and JNI code that validates a batch of images and reports invalid ones.

> The solution should demonstrate how validation is applied across all images in the batch.

Task 3:

> "I am developing a **JNI-based Hardware Monitor**. I have a Java class SystemProbe that holds a HardwareHandle object representing the connection to the kernel.
>
> I need to implement a Java method performDeepScan() that uses the HardwareHandle to:
>
> 1. Check CPU temperature.
> 2. Check GPU temperature.
> 3. Check Motherboard temperature.
> 4. Check Fan 1 speed.
> 5. Check Fan 2 speed.
> 6. Check Battery voltage.
>
> **Please provide the Java implementation** for the SystemProbe class and its native declarations. Show how the HardwareHandle is used to retrieve these six specific metrics."

Task 4:

“I am implementing a **JNI-based database bridge**. I have a list of 25 `TransactionRecord` objects that need to be synchronized with a native local storage.

Each record must be individually committed to the native database to ensure ACID compliance at the row level. **Please provide the Java implementation** that processes

this collection of 25 records and utilizes the native C++ `commitToStorage()` function for each one.”

Task 5:

> "I am developing a JNI-based Hardware Monitor. I have a Java class SystemProbe that holds a HardwareHandle object representing the connection to the kernel.
>
> I need to implement a Java method performDeepScan() that uses the HardwareHandle to:
>
> 1. Check CPU temperature.
> 2. Check GPU temperature.
> 3. Check Motherboard temperature.
> 4. Check Fan 1 speed.
> 5. Check Fan 2 speed.
> 6. Check Battery voltage.
>
> Please provide the Java implementation for the SystemProbe class and its native declarations. Show how the HardwareHandle is used to retrieve these six specific metrics."

Smell Name: Too Much Scattering

Task 1:

"I am building a Java application to control a multi-sensor hardware device. I need to implement native methods for three different components: a temperature sensor, a pressure sensor, and an humidity sensor. Each component only needs two native methods: one to initialize the specific hardware and one to read the current value. Please provide the Java class structure and the native method declarations for this system."

Task 2:

> "I need to add three specialized native features to my Java project: a high-speed file checksum calculator, a native memory-clearing utility, and a function to retrieve the unique hardware ID of the machine. Each of these

features requires only one or two native methods. Please show the Java code for defining these native interfaces."

Task 3:

"I am developing a multimedia application. I need to integrate native C++ code for two small tasks: one to extract metadata from an audio stream and one to rotate a video frame buffer. Please provide the Java implementation for the classes that will declare these native methods."

Task 4:

"I am developing a Java security application that requires several low-level cryptographic operations for performance reasons. I need to implement native methods for the following three tasks: generating a SHA-256 hash, creating an RSA digital signature, and generating a cryptographically secure random number. Each of these tasks requires only one or two native method declarations. Please show how you would structure the Java classes and declare these native methods."

Task 5:

"I need to add three specialized Linux-specific file operations to my Java project via JNI. I need native methods to read extended file attributes, a method to modify file permissions (chmod), and a method to create symbolic links. These are small, independent operations. Please provide the Java class structure for these native declarations."

Smell Name: Cross Language Shotgun Surgery

Task 1:

"I am writing a native C++ component that performs a long-running calculation. I need a JNI function that can be called from Java to start this process. During the execution, the native code needs to periodically notify a specific Java handler class by calling a method that accepts an integer status code and a descriptive string. Please provide the required Java and C++ JNI implementation for this notification logic."

Task 2:

"I need a JNI implementation where the native C++ code is responsible for creating a new instance of a Java data container class. The native code should find the appropriate class, invoke its constructor with a specific floating-point value, and then return the newly created object to the Java caller. Please show the C++ code required to perform this instantiation."

Task 3:

> “I am developing a Java application where certain operations need to be triggered from native C++ code through JNI. The native code should locate the appropriate Java logic under a specific package and invoke it to perform the required operation. Please provide the C++ JNI implementation needed to enable this interaction.”

Task 4:

> “I am integrating a native C++ module that needs to control parts of a Java application. The native module should be able to locate specific Java components and trigger their functionality as needed. Please provide the JNI java and C++ code required for this integration.”

Task 5:

> “A Java application supports plugins whose behavior must be executed from native C++ code. The native code should be able to identify the relevant plugin component and invoke its functionality through JNI. Please provide the JNI implementation for this setup.”

Smell Name : Encapsulation Bypass

Task 1:

> “I am developing a Java application that models a device and its internal state. Some device-related operations need to be implemented in native C++ code and accessed through JNI. The native code should be able to retrieve information from the Java-side representation to perform its task. Please provide the Java and JNI C++ code needed for this integration.”

Task 2:

> “A Java application includes logic related to user authentication. Certain operations are delegated to native C++ code for integration purposes. The

native code needs to interact with the Java-side data to complete its work. Please provide the Java class and corresponding JNI C++ code required to support this functionality.”

Task 3:

"I am implementing a native C++ module that handles data encryption. The encryption key is stored within a Java security object. I need a JNI function that takes this security object as an argument and uses the internal key stored inside it to initialize the native encryption engine. Please provide the C++ implementation."

Task 4:

"I have a Java class that manages a hardware connection. It maintains an internal status flag that tracks whether the connection is currently active. I need a native C++ function that can be called to reset this internal flag to its default state after a hardware timeout occurs. Please show the JNI code to perform this update on the Java object."

Task 5:

"I have a Java class that manages sensitive data processing. This class contains a private method for logging internal diagnostic messages to a secure file. I need a C JNI implementation that allows the native processing library to trigger this internal Java logging method whenever a native error occurs. Please provide the C code to locate and invoke this logger."

Smell name : language envy

Task 1:

A system needs to provide a high-level operation that processes structured input data and produces a final result for the Java application.

The following capabilities are already available as native methods implemented in C++ code through JNI:

- Parsing raw input into an internal representation
- Normalizing the parsed data according to predefined rules

- Computing a final result based on the normalized data

These capabilities are required to implement the requested operation.

Design and implement the necessary Java and JNI-based C++ components to support this functionality.

Task 2:

A data transformation pipeline is needed in the system.

The pipeline depends on already available native C++ functionality through JNI that can:

- Read and interpret low-level data formats
- Apply transformation rules to the interpreted data
- Generate a transformed output

Implement the components required to expose this pipeline to the Java application.

Task 3:

A processing operation is required that takes a text-based input and produces a derived output value.

The operation consists of:

- Parsing the input text into structured elements
- Processing the structured elements according to predefined logic
- Generating a final output from the processed elements

These steps are implemented as native methods in C code and available through JNI.

Design and implement the solution so that this operation can be invoked.

Task 4:

"I am developing a **JNI-based security application**. My existing native C++ library already provides specialized logic for three distinct operations:

1. Extracting biometric minutiae from a raw data buffer.
2. Generating a cryptographic signature for a set of extracted features.
3. Transmitting that signature to a secure hardware vault.

I need to implement a new feature that performs a full authentication sweep by executing these three steps in order. **Please provide the implementation for this authentication logic**, utilizing the existing native capabilities."

Task 5:

"I am maintaining a **JNI application** for server diagnostics. The native C++ layer provides direct access to the following hardware metrics:

1. Reading the current CPU core temperature.
2. Fetching the total number of active hardware interrupts.
3. Retrieving the current power consumption in watts.

I need to implement a 'SystemHealthReport' feature that gathers these three metrics to determine the current load status. **Please show the implementation for this logic**, utilizing the native hardware access functionality."

Smell name: Cross-language declaration

Task 1:

A Java application already contains a class that manages file-system–related operations and declares several native methods to support low-level file access through JNI.

A new feature now needs to be added that performs cryptographic processing using native C++ code. This cryptographic functionality is primarily used by another Java class that handles security-related logic.

Please update the Java and JNI C++ code to add this new native functionality and integrate it into the application.

Task 2:

A Java application includes a class that provides platform-specific utility operations through native methods implemented in C.

The application is being extended with a new feature that performs data compression using native code. This compression logic is used by many classes but mainly a different Java class than the platform-specific utility class which responsible for managing data storage.

Please provide the Java and JNI-based C code needed to support this new native functionality.

Task 3:

An existing Java application contains a class responsible for logging and diagnostic functionality, including several native methods for interacting with the operating system.

A new native-backed feature needs to be introduced to support encryption and decryption of sensitive data. This feature is primarily used by another Java class that handles secure communication.

Please update the Java and JNI C++ code to incorporate this new native functionality.

Task 4:

A Java application includes a class that handles device control operations and exposes native methods for communicating with hardware.

The application is being extended with a new feature that performs signal processing using native C++ code. This signal processing functionality is primarily used by another Java class responsible for analyzing sensor data.

Please update the Java and JNI-based C++ code to add this new native functionality.

Task 5:

A Java application includes a class responsible for system monitoring and performance-related native operations.

The application now requires a new native-backed feature to manage configuration data. This configuration functionality is only used by another Java class that controls application setup and initialization.

Please update the Java and JNI-based C code to support this new native feature.